

FL Anomaly Detection — Adaptive Aggregation for Network Intrusion Detection

Complete Debugged Version — Run cells 1 to 9 in order.

Cell	Purpose
1	Install packages
2	Mount Drive + fix paths
3	Download + preprocess NSL-KDD
4	Visualise IID vs Non-IID partitioning
5	Phase 1 — quick validation (FedAvg vs Adaptive, 5 clients, 50 rounds)
6	Phase 2 — full sweep (4 strategies × 4 conditions = 16 experiments)
7	Phase 3 — UNSW-NB15 cross-dataset + extra seed
8	Generate all research graphs from REAL results only
9	Print full results table + save CSV
10	Save everything to Google Drive

```
# =====
# CELL 1 — Install Dependencies
# =====
import subprocess, sys, torch

for pkg in ['scikit-learn', 'seaborn', 'pyyaml', 'tqdm', 'requests', 'pandas']:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', pkg])

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'✅ Packages installed | Device: {device}')
if device == 'cuda':
    print(f'    GPU: {torch.cuda.get_device_name(0)}')
    print(f'    VRAM: {torch.cuda.get_device_properties(0).total_memory/1e9:.1f} GB')
```

```
✅ Packages installed | Device: cuda
GPU: Tesla T4
VRAM: 15.6 GB
```

```
# =====
# CELL 2 — Mount Drive + Fix Paths
# =====
import os, sys, shutil
from google.colab import drive
drive.mount('/content/drive')

def find_root():
    candidates = [
        '/content/fl_anomaly_detection/fl_anomaly_detection',
        '/content/fl_anomaly_detection',
        '/content/drive/MyDrive/fl_anomaly_detection/fl_anomaly_detection',
        '/content/drive/MyDrive/fl_anomaly_detection',
    ]
    for p in candidates:
        if os.path.exists(os.path.join(p, 'data', 'download.py')):
            return p
    return None

REPO_ROOT = find_root()
if REPO_ROOT is None:
    raise RuntimeError(
        'Project not found.\n'
        'Make sure fl_anomaly_detection is in Google Drive.\n'
        'Expected to find: data/download.py inside the project folder.'
    )

if REPO_ROOT not in sys.path:
    sys.path.insert(0, REPO_ROOT)
os.chdir(REPO_ROOT)

for d in ['results/logs', 'results/plots', 'results/checkpoints',
          'data/raw/nsl_kdd', 'data/processed/nsl_kdd',
          'data/raw/unswnb15', 'data/processed/unswnb15']:
    os.makedirs(os.path.join(REPO_ROOT, d), exist_ok=True)

print(f'✅ REPO_ROOT = {REPO_ROOT}')
print(f'    Files: {sorted(os.listdir(REPO_ROOT))}')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
✅ REPO_ROOT = /content/drive/MyDrive/fl_anomaly_detection
Files: ['README.md', 'aggregation', 'configs', 'data', 'evaluation', 'federation', 'main.py', 'models', 'notebooks', 'pha
```

```
# =====
# CELL 3 – Download + Preprocess NSL-KDD
# =====
import os, sys, numpy as np

# Re-ensure path
for p in sys.path:
    if 'fl_anomaly_detection' in p and os.path.exists(os.path.join(p, 'data', 'download.py')):
        REPO_ROOT = p; break
os.chdir(REPO_ROOT)

from data.download import download_nsl_kdd
from data.preprocess import preprocess_nsl_kdd, load_processed

print('Downloading NSL-KDD...')
download_nsl_kdd()
print('Preprocessing...')
X_train, X_test, y_train, y_test, y_bin_train, y_bin_test = preprocess_nsl_kdd()

print(f'\n📊 NSL-KDD:')
print(f' Train : {X_train.shape} | Normal={np.sum(y_bin_train==0):,} Attack={np.sum(y_bin_train==1):,}')
print(f' Test  : {X_test.shape} | Normal={np.sum(y_bin_test==0):,} Attack={np.sum(y_bin_test==1):,}')
print(f' dtype : X={X_train.dtype} y={y_bin_train.dtype}')
print('✅ Data ready')
```

```
Downloading NSL-KDD...
[NSL-KDD] Downloading train...
[NSL-KDD] Saved to data/raw/nsl_kdd/KDDTrain+.txt
[NSL-KDD] Downloading test...
[NSL-KDD] Saved to data/raw/nsl_kdd/KDDTest+.txt
[NSL-KDD] Train: 125,973 rows | Test: 22,544 rows
Preprocessing...
[NSL-KDD] Preprocessed. X_train=(125973, 41), X_test=(22544, 41)
```

```
📊 NSL-KDD:
Train : (125973, 41) | Normal=67,343 Attack=58,630
Test  : (22544, 41) | Normal=9,711 Attack=12,833
dtype : X=float32 y=int64
✅ Data ready
```

```
# =====
# CELL 4 – Visualise IID vs Non-IID
# =====
import matplotlib.pyplot as plt
import numpy as np
from data.partition import get_partitions

NUM = 5
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
CASES = [('IID', 'iid', None), ('Non-IID α=0.5', 'non_iid', 0.5), ('Non-IID α=0.1', 'non_iid', 0.1)]
COLS = ['#4CAF50', '#F44336', '#2196F3', '#FF9800', '#9C27B0']
LABELS = ['Normal', 'DoS', 'Probe', 'R2L', 'U2R']

for ax, (title, strat, alpha) in zip(axes, CASES):
    parts = get_partitions(X_train, y_train, NUM, strat, alpha or 0.5)
    data = np.zeros((NUM, 5))
    for i, (_, yc) in enumerate(parts):
        for c in range(5): data[i, c] = np.sum(np.array(yc)==c)
    pct = data / np.maximum(data.sum(1, keepdims=True), 1) * 100
    bot = np.zeros(NUM)
    for c in range(5):
        ax.bar(range(NUM), pct[:, c], bottom=bot, color=COLS[c], label=LABELS[c], alpha=0.85)
        bot += pct[:, c]
    ax.set_title(title, fontweight='bold')
    ax.set_xlabel('Client'); ax.set_ylabel('Class %')
    ax.set_xticks(range(NUM)); ax.set_xticklabels([f'C{i}' for i in range(NUM)])

axes[2].legend(loc='upper right', fontsize=8)
plt.suptitle('Data Distribution Across Clients', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('results/plots/0_data_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ Graph 0 saved')
```

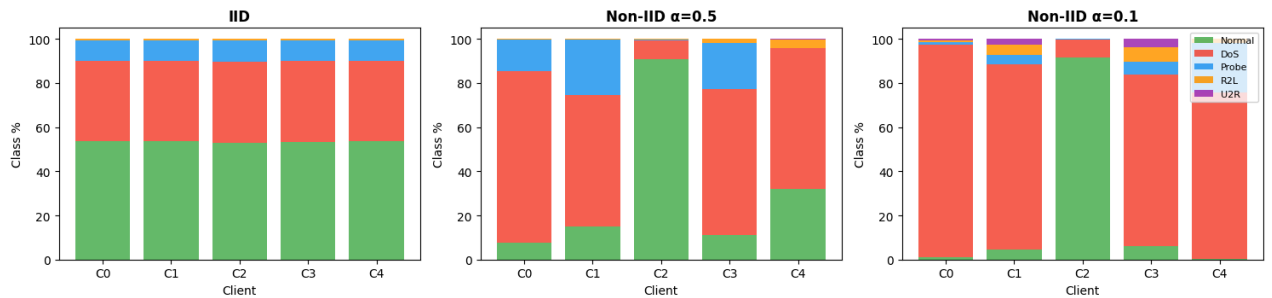
[IID] 5 clients | avg 25194 samples each
 [Non-IID $\alpha=0.5$] 5 clients | min=7505 max=63719 avg=25194 samples

Client	Class0	Class1	Class2	Class3	Class4
0	7.6%	77.7%	14.5%	0.1%	0.1%
1	15.1%	59.6%	25.1%	0.1%	0.1%
2	90.8%	8.5%	0.5%	0.2%	0.0%
3	11.2%	66.1%	20.8%	1.8%	0.0%
4	32.2%	63.5%	0.3%	3.9%	0.1%

[Non-IID $\alpha=0.1$] 5 clients | min=167 max=73216 avg=25194 samples

Client	Class0	Class1	Class2	Class3	Class4
0	0.9%	96.5%	0.9%	0.9%	0.6%
1	4.5%	83.9%	4.5%	4.5%	2.7%
2	91.7%	8.1%	0.1%	0.0%	0.0%
3	6.0%	77.8%	6.0%	6.6%	3.6%
4	0.4%	75.4%	22.4%	1.9%	0.0%

Data Distribution Across Clients



✓ Graph 0 saved

```
# =====
# CELL 5 - Phase 1: FedAvg vs Adaptive (5 clients, 50 rounds, quick test)
# =====

import copy, yaml, torch, importlib
import federation.simulation
importlib.reload(federation.simulation)
from federation.simulation import FLSimulation
from data.partition import get_partitions
from data.preprocess import load_processed

with open('configs/base_config.yaml') as f:
    BASE = yaml.safe_load(f)

BASE['federation']['num_clients'] = 5
BASE['federation']['num_rounds'] = 50
BASE['experiment']['device'] = 'cuda' if torch.cuda.is_available() else 'cpu'
BASE['results'] = {
    'log_dir': 'results/logs',
    'plot_dir': 'results/plots',
    'checkpoint_dir': 'results/checkpoints',
    'save_every_n_rounds': 25
}

X_train, X_test, _, _, y_bin_train, y_bin_test = load_processed('nsl_kdd')
phase1_results = {}

for strategy in ['fedavg', 'adaptive']:
    for (part, alpha) in [(('iid', None), ('non_iid', 0.1))]:
        label = 'IID' if part == 'iid' else 'NonIID_a0.1'
        exp_name = f'{strategy}_{label}'
        print(f'\n{"*" * 45}\n{exp_name}\n{"*" * 45}')

        cfg = copy.deepcopy(BASE)
        cfg['experiment']['name'] = exp_name
        cfg['aggregation']['strategy'] = strategy
        cfg['partitioning']['strategy'] = part
        cfg['partitioning']['dirichlet_alpha'] = float(alpha or 0.5)

        parts = get_partitions(X_train, y_bin_train, 5, part, float(alpha or 0.5))
        try:
            sim = FLSimulation(cfg)
            sim.setup(parts, X_test, y_bin_test)
            res = sim.run()
            phase1_results[exp_name] = res
            m = res['final_metrics']
            print(f' ✓ AUC={m["auc_roc"]:.4f} F1={m["f1"]:.4f} DR={m["dr"]:.4f} FAR={m["far"]:.4f} Conv={res.get("conver
        except Exception as e:
```

```

import traceback
print(f' ✖ {e}'); traceback.print_exc()

print(f'\n✅ Phase 1 done: {len(phase1_results)}/4')

INFO:federation.simulation:Round 45/50 | Loss=0.2610 | AUC=0.9477 | alpha=[0.204 0.194 0.205 0.199]...
10:11:55 | INFO | federation.simulation | Early stopping at round 45
INFO:federation.simulation:Early stopping at round 45
10:11:55 | INFO | federation.simulation | Done in 632.6s
INFO:federation.simulation:Done in 632.6s
10:11:55 | INFO | federation.simulation | Saved → results/logs/adaptive_IID_results.json
INFO:federation.simulation:Saved → results/logs/adaptive_IID_results.json
✅ AUC=0.9477 F1=0.8338 DR=0.7459 FAR=0.0498 Conv=35

=====
adaptive_NonIID_a0.1
=====
[Non-IID  $\alpha=0.1$ ] 5 clients | min=284 max=73427 avg=25194 samples
Client    Class0    Class1
0         3.2%    96.8%
1         1.4%    98.6%
2         0.0%   100.0%
3        86.8%    13.2%
4        97.6%     2.4%
10:11:55 | INFO | federation.simulation | Using GPU: Tesla T4
INFO:federation.simulation:Using GPU: Tesla T4
10:11:55 | INFO | federation.simulation | Model params: 4049 | input_dim: 41
INFO:federation.simulation:Model params: 4049 | input_dim: 41
10:11:55 | INFO | federation.simulation | Clients: 5
INFO:federation.simulation:Clients: 5
10:11:55 | INFO | federation.simulation | Aggregation strategy: adaptive
INFO:federation.simulation:Aggregation strategy: adaptive
10:11:55 | INFO | federation.simulation | =====
INFO:federation.simulation:=====
10:11:55 | INFO | federation.simulation | FL start | rounds=50 clients=5 strategy=adaptive
INFO:federation.simulation:FL start | rounds=50 clients=5 strategy=adaptive
10:11:55 | INFO | federation.simulation | =====
INFO:federation.simulation:=====
10:13:06 | INFO | federation.simulation | Round 5/50 | Loss=0.2620 | AUC=0.9226 | alpha=[0.142 0.136 0.137 0.38 ]...
INFO:federation.simulation:Round 5/50 | Loss=0.2620 | AUC=0.9226 | alpha=[0.142 0.136 0.137 0.38 ]...
10:14:16 | INFO | federation.simulation | Round 10/50 | Loss=0.2027 | AUC=0.9382 | alpha=[0.142 0.143 0.144 0.38 ]...
INFO:federation.simulation:Round 10/50 | Loss=0.2027 | AUC=0.9382 | alpha=[0.142 0.143 0.144 0.38 ]...
10:15:25 | INFO | federation.simulation | Round 15/50 | Loss=0.1896 | AUC=0.9425 | alpha=[0.141 0.142 0.142 0.385]...
INFO:federation.simulation:Round 15/50 | Loss=0.1896 | AUC=0.9425 | alpha=[0.141 0.142 0.142 0.385]...
10:16:34 | INFO | federation.simulation | Round 20/50 | Loss=0.1811 | AUC=0.9447 | alpha=[0.142 0.141 0.145 0.386]...
INFO:federation.simulation:Round 20/50 | Loss=0.1811 | AUC=0.9447 | alpha=[0.142 0.141 0.145 0.386]...
10:17:44 | INFO | federation.simulation | Round 25/50 | Loss=0.1540 | AUC=0.9461 | alpha=[0.145 0.144 0.144 0.39 ]...
INFO:federation.simulation:Round 25/50 | Loss=0.1540 | AUC=0.9461 | alpha=[0.145 0.144 0.144 0.39 ]...
10:18:54 | INFO | federation.simulation | Round 30/50 | Loss=0.1442 | AUC=0.9457 | alpha=[0.146 0.144 0.146 0.393]...
INFO:federation.simulation:Round 30/50 | Loss=0.1442 | AUC=0.9457 | alpha=[0.146 0.144 0.146 0.393]...
10:20:05 | INFO | federation.simulation | Round 35/50 | Loss=0.1558 | AUC=0.9459 | alpha=[0.148 0.147 0.141 0.393]...
INFO:federation.simulation:Round 35/50 | Loss=0.1558 | AUC=0.9459 | alpha=[0.148 0.147 0.141 0.393]...
10:21:15 | INFO | federation.simulation | Round 40/50 | Loss=0.1516 | AUC=0.9459 | alpha=[0.149 0.146 0.148 0.397]...
INFO:federation.simulation:Round 40/50 | Loss=0.1516 | AUC=0.9459 | alpha=[0.149 0.146 0.148 0.397]...
10:21:43 | INFO | federation.simulation | Early stopping at round 42
INFO:federation.simulation:Early stopping at round 42
10:21:43 | INFO | federation.simulation | Done in 587.1s
INFO:federation.simulation:Done in 587.1s
10:21:43 | INFO | federation.simulation | Saved → results/logs/adaptive_NonIID_a0.1_results.json
INFO:federation.simulation:Saved → results/logs/adaptive_NonIID_a0.1_results.json
✅ AUC=0.9452 F1=0.8087 DR=0.7017 FAR=0.0498 Conv=32

✅ Phase 1 done: 4/4

```

```

#visualization of phase 5 results
import pandas as pd
import matplotlib.pyplot as plt

# Convert results into a dataframe
rows = []
for name, res in phase1_results.items():
    m = res['final_metrics']
    rows.append({
        'Experiment': name,
        'AUC': m['auc_roc'],
        'F1': m['f1'],
        'DR': m['dr'],
        'FAR': m['far']
    })

df = pd.DataFrame(rows)

# Set experiment names as index
df.set_index('Experiment', inplace=True)

print(df) # optional: see values

```

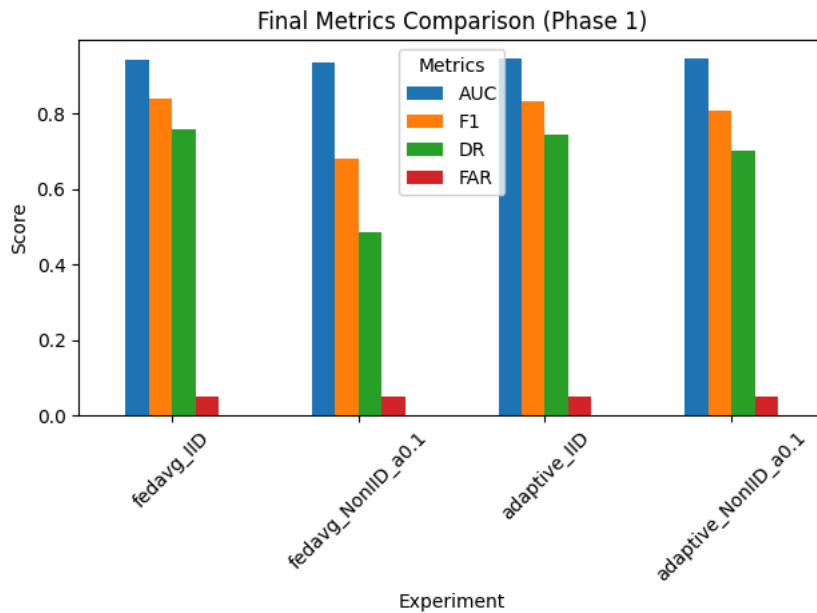
```
# Plot all metrics together
plt.figure()
df.plot(kind='bar')

plt.title("Final Metrics Comparison (Phase 1)")
plt.xlabel("Experiment")
plt.ylabel("Score")
plt.xticks(rotation=45)
plt.legend(title="Metrics")
plt.tight_layout()

plt.show()
```

Experiment	AUC	F1	DR	FAR
fedavg_IID	0.942658	0.841778	0.760072	0.050046
fedavg_NonIID_a0.1	0.935252	0.680897	0.486792	0.049634
adaptive_IID	0.947709	0.833841	0.745890	0.049840
adaptive_NonIID_a0.1	0.945181	0.808711	0.701707	0.049840

<Figure size 640x480 with 0 Axes>



```
import sys, importlib

# force reload of adaptive module
if "aggregation.adaptive" in sys.modules:
    importlib.reload(sys.modules["aggregation.adaptive"])

# optional: clear compiled cache (safe)
import os
cache_path = "aggregation/__pycache__"
if os.path.exists(cache_path):
    import shutil
    shutil.rmtree(cache_path)

print("✅ adaptive.py reloaded cleanly")
```

✅ adaptive.py reloaded cleanly

```
import federation.simulation
importlib.reload(federation.simulation)

from federation.simulation import FLSimulation
```

```
# =====
# CELL 6 – Phase 2: Full sweep (4 strategies × 4 conditions = 16 runs)
# =====
```

```
import copy, yaml, torch, json, os, importlib
import numpy as np

import federation.simulation
importlib.reload(federation.simulation)

from federation.simulation import FLSimulation
from data.partition import get_partitions
```

```

from data.preprocess import load_processed

# ☒ NEW – reset FedAdam state
from aggregation.adaptive import reset_fedadam_state

# -----
# Load base config
# -----
with open('configs/base_config.yaml') as f:
    BASE = yaml.safe_load(f)

BASE['federation']['num_clients'] = 10
BASE['federation']['num_rounds'] = 100

BASE['experiment']['device'] = 'cuda' if torch.cuda.is_available() else 'cpu'

BASE['results'] = {
    'log_dir':          'results/logs',
    'plot_dir':         'results/plots',
    'checkpoint_dir':   'results/checkpoints',
    'save_every_n_rounds': 10
}

# -----
# Data
# -----
X_train, X_test, _, _, y_bin_train, y_bin_test = load_processed('nsl_kdd')

# -----
# EXPERIMENT CONFIG
# -----
STRATEGIES = ['fedavg', 'fedprox', 'fedadam', 'adaptive']

CONDITIONS = [
    ('iid', None),
    ('non_iid', 1.0),
    ('non_iid', 0.5),
    ('non_iid', 0.1)
]

TOTAL = len(STRATEGIES) * len(CONDITIONS)

phase2_results = {}
done = 0

# -----
# MAIN LOOP
# -----
for strategy in STRATEGIES:
    for (part, alpha) in CONDITIONS:

        done += 1

        cond = 'IID' if part == 'iid' else f'NonIID_a{alpha}'
        exp_name = f'{strategy}_{cond}'

        print(f'\n{"="*60}')
        print(f'[{done}/{TOTAL}] {exp_name}')
        print(f'{"="*60}')

        cfg = copy.deepcopy(BASE)

        cfg['experiment']['name'] = exp_name
        cfg['aggregation']['strategy'] = strategy

        cfg['partitioning']['strategy'] = part
        cfg['partitioning']['dirichlet_alpha'] = float(alpha if alpha is not None else 0.5)

        # -----
        # partitions
        # -----
        parts = get_partitions(
            X=X_train,
            y=y_bin_train,
            num_clients=10,
            strategy=part,
            alpha=float(alpha if alpha is not None else 0.5)
        )

```

```

try:
    # ✅ CRITICAL – reset FedAdam per experiment
    if strategy == "fedadam":
        reset_fedadam_state()

    sim = FLSimulation(cfg)
    sim.setup(parts, X_test, y_bin_test)

    res = sim.run()
    phase2_results[exp_name] = res

    m = res['final_metrics']
    cv = res.get('convergence_round', 100)

    print(
        f' ✅ AUC={m["auc_roc"]:.4f} | '
        f'F1={m["f1"]:.4f} | '
        f'DR={m["dr"]:.4f} | '
        f'FAR={m["far"]:.4f} | '
        f'Conv={cv}'
    )

except Exception as e:
    import traceback
    print(f' ❌ FAILED: {e}')
    traceback.print_exc()
    continue

# -----
# SAVE RESULTS
# -----
os.makedirs('results', exist_ok=True)

with open('results/phase2_all_results.json', 'w') as f:
    json.dump(
        {k: v['final_metrics'] for k, v in phase2_results.items()},
        f,
        indent=2
    )

print(f'\n✅ Phase 2 complete: {len(phase2_results)}/{TOTAL}')
print('📁 Saved → results/phase2_all_results.json')

```

```

INFO:federation.simulation:Round 5/100 | Loss=0.2605 | AUC=0.9327
11:30:37 | INFO | federation.simulation | Round 10/100 | Loss=0.2287 | AUC=0.9392
INFO:federation.simulation:Round 10/100 | Loss=0.2287 | AUC=0.9392
11:31:45 | INFO | federation.simulation | Round 15/100 | Loss=0.2210 | AUC=0.9382
INFO:federation.simulation:Round 15/100 | Loss=0.2210 | AUC=0.9382
11:32:54 | INFO | federation.simulation | Round 20/100 | Loss=0.2075 | AUC=0.9380
INFO:federation.simulation:Round 20/100 | Loss=0.2075 | AUC=0.9380
11:33:07 | INFO | federation.simulation | Early stopping at round 21
INFO:federation.simulation:Early stopping at round 21
11:33:07 | INFO | federation.simulation | Done in 287.0s
INFO:federation.simulation:Done in 287.0s
11:33:08 | INFO | federation.simulation | Saved → results/logs/adaptive_NonIID_a0.1_results.json
INFO:federation.simulation:Saved → results/logs/adaptive_NonIID_a0.1_results.json
✅ AUC=0.9370 | F1=0.8437 | DR=0.7635 | FAR=0.0500 | Conv=11

```

✅ Phase 2 complete: 16/16
 📁 Saved → results/phase2_all_results.json

```

#visualization of above results
import json
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load the data
with open('results/phase2_all_results.json', 'r') as f:
    raw_data = json.load(f)

# 2. Transform the dictionary into a clean DataFrame
rows = []
for key, metrics in raw_data.items():
    # Split key (e.g., 'fedprox_NonIID_a1.0') into strategy and condition
    parts = key.split('_')
    strategy = parts[0]
    condition = "_".join(parts[1:])

    metrics['Strategy'] = strategy
    metrics['Condition'] = condition
    rows.append(metrics)

df = pd.DataFrame(rows)

# Define the order for a cleaner look
strategy_order = ['fedavg', 'fedprox', 'fedadam', 'adaptive']
condition_order = ['IID', 'NonIID_a1.0', 'NonIID_a0.5', 'NonIID_a0.1']

# 3. Create the Visualizations
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# --- Plot A: AUC Heatmap (Strategy vs Condition) ---
pivot_auc = df.pivot(index="Strategy", columns="Condition", values="auc_roc")
pivot_auc = pivot_auc.reindex(index=strategy_order, columns=condition_order)

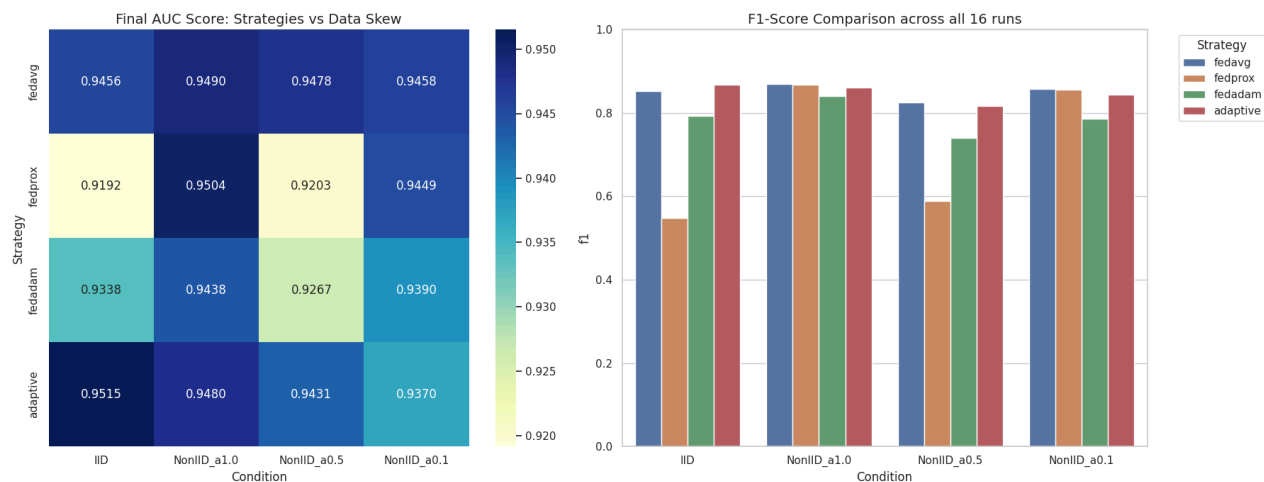
sns.heatmap(pivot_auc, annot=True, fmt=".4f", cmap="YlGnBu", ax=axes[0])
axes[0].set_title('Final AUC Score: Strategies vs Data Skew', fontsize=14)

# --- Plot B: Metric Comparison (Grouped Bar Chart) ---
sns.barplot(data=df, x='Condition', y='f1', hue='Strategy',
            hue_order=strategy_order, order=condition_order, ax=axes[1])
axes[1].set_title('F1-Score Comparison across all 16 runs', fontsize=14)
axes[1].set_ylim(0, 1.0)
axes[1].legend(title='Strategy', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.savefig('results/full_sweep_comparison.png', dpi=300)
plt.show()

# 4. (Optional) Print a summary table for your paper/report
print("\n--- Summary Table (Sorted by AUC) ---")
print(df[['Strategy', 'Condition', 'auc_roc', 'f1', 'dr']].sort_values(by='auc_roc', ascending=False).to_string(index=False)

```



--- Summary Table (Sorted by AUC) ---

Strategy	Condition	auc_roc	f1	dr
adaptive	IID	0.951513	0.867271	0.805268
fedprox	NonIID_a1.0	0.950412	0.868149	0.806826
fedavg	NonIID_a1.0	0.949034	0.869777	0.809398
adaptive	NonIID_a1.0	0.947960	0.861653	0.795293
fedavg	NonIID_a0.5	0.947756	0.825179	0.730461
fedavg	NonIID_a0.1	0.945816	0.857487	0.787345
fedavg	IID	0.945609	0.853130	0.780176
fedprox	NonIID_a0.1	0.944902	0.855551	0.784150
fedadam	NonIID_a1.0	0.943830	0.840019	0.756487
adaptive	NonIID_a0.5	0.943089	0.816003	0.714486
fedadam	NonIID_a0.1	0.939020	0.786150	0.662355
adaptive	NonIID_a0.1	0.937035	0.843716	0.763500
fedadam	IID	0.933767	0.792176	0.673030
fedadam	NonIID_a0.5	0.926737	0.740466	0.583885
fedprox	NonIID_a0.5	0.920295	0.588555	0.347230
fedprox	IID	0.919171	0.547997	0.290891

```
# =====
# CELL 7 – Phase 3: UNSW-NB15 + extra seed for statistical robustness
# 2 seeds x 4 strategies x 4 conditions = 32 runs
# =====

# =====
# CELL 7 – Phase 3: UNSW-NB15 + FedAdam + fixed download
# =====

import copy, yaml, torch, json, os, requests, importlib
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
import pandas as pd

import federation.simulation
importlib.reload(federation.simulation)

from federation.simulation import FLSimulation
from data.partition import get_partitions

# — 1. Download UNSW-NB15 (FIXED) —————
UNSW_DIR = 'data/raw/unsw_nb15'
TRAIN_PATH = f'{UNSW_DIR}/UNSW_NB15_train.csv'
TEST_PATH = f'{UNSW_DIR}/UNSW_NB15_test.csv'
os.makedirs(UNSW_DIR, exist_ok=True)

MIRRORS = [
    'https://raw.githubusercontent.com/ushukkla/nospammers/master/UNSW_NB15_training-set.csv',
    'https://raw.githubusercontent.com/ushukkla/nospammers/master/UNSW_NB15_testing-set.csv',
    'https://raw.githubusercontent.com/699-601/601-project/master/UNSW_NB15_training-set.csv',
    'https://raw.githubusercontent.com/699-601/601-project/master/UNSW_NB15_testing-set.csv',
]

def try_download(url, dest):
    try:
```

```

    r = requests.get(url, timeout=30)
    if r.status_code == 200:
        with open(dest, 'wb') as f:
            f.write(r.content)
        print(f' ✅ Downloaded: {dest}')
        return True
    except:
        pass
    return False

print('Downloading UNSW-NB15...')

# Download TRAIN
if not os.path.exists(TRAIN_PATH):
    for url in MIRRORS:
        if 'training' in url.lower():
            if try_download(url, TRAIN_PATH):
                break

# Download TEST
if not os.path.exists(TEST_PATH):
    for url in MIRRORS:
        if 'testing' in url.lower():
            if try_download(url, TEST_PATH):
                break

# Final check
if not os.path.exists(TRAIN_PATH) or not os.path.exists(TEST_PATH):
    raise FileNotFoundError(
        "UNSW-NB15 files missing.\n"
        "Download manually from Kaggle:\n"
        "https://www.kaggle.com/datasets/mrwellsdavid/unsw-nb15"
    )

# — 2. Preprocess —————
PROC_DIR = 'data/processed/unsw_nb15'
os.makedirs(PROC_DIR, exist_ok=True)

if os.path.exists(f'{PROC_DIR}/X_train.npy'):
    print('Loading preprocessed UNSW-NB15...')
    Xu_train = np.load(f'{PROC_DIR}/X_train.npy')
    Xu_test = np.load(f'{PROC_DIR}/X_test.npy')
    yu_bin_train = np.load(f'{PROC_DIR}/y_bin_train.npy')
    yu_bin_test = np.load(f'{PROC_DIR}/y_bin_test.npy')
else:
    print('Preprocessing UNSW-NB15...')
    train_df = pd.read_csv(TRAIN_PATH)
    test_df = pd.read_csv(TEST_PATH)

    for df in [train_df, test_df]:
        for c in ['id', 'attack_cat']:
            if c in df.columns:
                df.drop(columns=[c], inplace=True)

    y_tr = train_df['label'].values
    y_te = test_df['label'].values

    train_df.drop(columns=['label'], inplace=True)
    test_df.drop(columns=['label'], inplace=True)

    cat_cols = train_df.select_dtypes(include=['object']).columns.tolist()

    for col in cat_cols:
        le = LabelEncoder()
        combined = pd.concat([train_df[col], test_df[col]]).astype(str)
        le.fit(combined)
        train_df[col] = le.transform(train_df[col].astype(str))
        test_df[col] = le.transform(test_df[col].astype(str))

    scaler = StandardScaler()
    Xu_train = scaler.fit_transform(train_df.values.astype(np.float32))
    Xu_test = scaler.transform(test_df.values.astype(np.float32))

    yu_bin_train = (y_tr > 0).astype(int)
    yu_bin_test = (y_te > 0).astype(int)

    np.save(f'{PROC_DIR}/X_train.npy', Xu_train)
    np.save(f'{PROC_DIR}/X_test.npy', Xu_test)
    np.save(f'{PROC_DIR}/y_bin_train.npy', yu_bin_train)

```

```

np.save(f'{PROC_DIR}/y_bin_test.npy', yu_bin_test)

print(f'UNSW-NB15: Train={Xu_train.shape} Test={Xu_test.shape}')

# — 3. Config —————
with open('configs/base_config.yaml') as f:
    BASE3 = yaml.safe_load(f)

BASE3['federation']['num_clients'] = 10
BASE3['federation']['num_rounds'] = 100
BASE3['experiment']['device'] = 'cuda' if torch.cuda.is_available() else 'cpu'
BASE3['autoencoder']['input_dim'] = int(Xu_train.shape[1])

BASE3['results'] = {
    'log_dir': 'results/logs',
    'plot_dir': 'results/plots',
    'checkpoint_dir': 'results/checkpoints',
    'save_every_n_rounds': 25
}

# — 4. Experiments —————
SEEDS = [42, 123]
STRATEGIES3 = ['fedavg', 'fedprox', 'fedadam', 'adaptive']
CONDITIONS3 = [
    ('iid', None),
    ('non_iid', 1.0),
    ('non_iid', 0.5),
    ('non_iid', 0.1)
]

TOTAL3 = len(SEEDS) * len(STRATEGIES3) * len(CONDITIONS3)

phase3_results = {}
done3 = 0

# — 5. Run —————
for seed in SEEDS:
    for strategy in STRATEGIES3:
        for (part, alpha) in CONDITIONS3:

            done3 += 1

            cond = 'IID' if part == 'iid' else f'NonIID_a{alpha}'
            exp_name = f'unsw_{strategy}_{cond}_s{seed}'

            print(f'\n[{done3}/{TOTAL3}] {exp_name}')

            cfg = copy.deepcopy(BASE3)
            cfg['experiment']['name'] = exp_name
            cfg['experiment']['seed'] = seed
            cfg['aggregation']['strategy'] = strategy

            cfg['partitioning']['strategy'] = part
            cfg['partitioning']['dirichlet_alpha'] = float(alpha if alpha is not None else 0.5)

            parts = get_partitions(
                X=Xu_train,
                y=yu_bin_train,
                num_clients=10,
                strategy=part,
                alpha=float(alpha if alpha is not None else 0.5),
                seed=seed
            )

            try:
                if torch.cuda.is_available():
                    torch.cuda.empty_cache()

                sim = FLSimulation(cfg)
                sim.setup(parts, Xu_test, yu_bin_test)

                res = sim.run()
                phase3_results[exp_name] = res

                m = res['final_metrics']
                cv = res.get('convergence_round', 100)

                print(
                    f' ✅ AUC={m["auc_roc"]:.4f} | '

```

```

        f'F1={m["f1"]:.4f} | '
        f'DR={m["dr"]:.4f} | '
        f'FAR={m["far"]:.4f} | '
        f'Conv={cv}'
    )

except Exception as e:
    import traceback
    print(f' ✖ {e}')
    traceback.print_exc()
    continue

# — 6. Save —————
os.makedirs('results', exist_ok=True)

with open('results/phase3_raw_results.json', 'w') as f:
    json.dump(
        {k: v['final_metrics'] for k, v in phase3_results.items()},
        f,
        indent=2
    )

print(f'\n✅ Phase 3 complete: {len(phase3_results)}/{TOTAL3}')
print('💾 Saved → results/phase3_raw_results.json')
```

INFO:federation.simulation:Saved → results/logs/unsu_adaptive_NonIID_a0.5_s123_results.json
 ✅ AUC=0.7131 | F1=0.5009 | DR=0.2976 | FAR=0.0500 | Conv=5

[32/32] unsu_adaptive_NonIID_a0.1_s123
 [Non-IID $\alpha=0.1$] 10 clients | min=20 max=44051 avg=8233 samples

Client	Class0	Class1
0	100.0%	0.0%
1	99.7%	0.3%
2	99.3%	0.7%
3	50.0%	50.0%
4	98.6%	1.4%
5	86.5%	13.5%
6	50.0%	50.0%
7	0.0%	100.0%
8	1.6%	98.4%
9	1.7%	98.3%

16:09:49 | INFO | federation.simulation | Using CPU
 INFO:federation.simulation:Using CPU
 16:09:49 | INFO | federation.simulation | Model params: 4114 | input_dim: 42
 INFO:federation.simulation:Model params: 4114 | input_dim: 42
 16:09:49 | INFO | federation.simulation | Clients: 10
 INFO:federation.simulation:Clients: 10
 16:09:49 | INFO | federation.simulation | Aggregation strategy: adaptive
 INFO:federation.simulation:Aggregation strategy: adaptive
 16:09:49 | INFO | federation.simulation | =====
 INFO:federation.simulation:=====

16:09:49 | INFO | federation.simulation | FL start | rounds=100 clients=10 strategy=adaptive
 INFO:federation.simulation:FL start | rounds=100 clients=10 strategy=adaptive
 16:09:49 | INFO | federation.simulation | =====
 INFO:federation.simulation:=====

16:10:28 | INFO | federation.simulation | Round 5/100 | Loss=0.3078 | AUC=0.6916
 INFO:federation.simulation:Round 5/100 | Loss=0.3078 | AUC=0.6916
 16:11:08 | INFO | federation.simulation | Round 10/100 | Loss=0.2525 | AUC=0.7161
 INFO:federation.simulation:Round 10/100 | Loss=0.2525 | AUC=0.7161
 16:11:49 | INFO | federation.simulation | Round 15/100 | Loss=0.2447 | AUC=0.7199
 INFO:federation.simulation:Round 15/100 | Loss=0.2447 | AUC=0.7199
 16:12:27 | INFO | federation.simulation | Round 20/100 | Loss=0.2277 | AUC=0.7259
 INFO:federation.simulation:Round 20/100 | Loss=0.2277 | AUC=0.7259
 16:13:08 | INFO | federation.simulation | Round 25/100 | Loss=0.2182 | AUC=0.7232
 INFO:federation.simulation:Round 25/100 | Loss=0.2182 | AUC=0.7232
 16:13:48 | INFO | federation.simulation | Round 30/100 | Loss=0.2218 | AUC=0.7317
 INFO:federation.simulation:Round 30/100 | Loss=0.2218 | AUC=0.7317
 16:14:27 | INFO | federation.simulation | Round 35/100 | Loss=0.2119 | AUC=0.7271
 INFO:federation.simulation:Round 35/100 | Loss=0.2119 | AUC=0.7271
 16:15:08 | INFO | federation.simulation | Round 40/100 | Loss=0.2159 | AUC=0.7281
 INFO:federation.simulation:Round 40/100 | Loss=0.2159 | AUC=0.7281
 16:15:49 | INFO | federation.simulation | Round 45/100 | Loss=0.2052 | AUC=0.7282
 INFO:federation.simulation:Round 45/100 | Loss=0.2052 | AUC=0.7282
 16:16:05 | INFO | federation.simulation | Early stopping at round 47
 INFO:federation.simulation:Early stopping at round 47
 16:16:05 | INFO | federation.simulation | Done in 375.6s
 INFO:federation.simulation:Done in 375.6s
 16:16:05 | INFO | federation.simulation | Saved → results/logs/unsu_adaptive_NonIID_a0.1_s123_results.json
 INFO:federation.simulation:Saved → results/logs/unsu_adaptive_NonIID_a0.1_s123_results.json
 ✅ AUC=0.7292 | F1=0.5710 | DR=0.3939 | FAR=0.0500 | Conv=37

✅ Phase 3 complete: 32/32
 💾 Saved → results/phase3_raw_results.json

```

import pandas as pd
import matplotlib.pyplot as plt
```

```

import seaborn as sns
import json
import os
import re
import numpy as np
from google.colab import drive

# 1. Mount Google Drive
try:
    drive.mount('/content/drive', force_remount=True)
except:
    pass

def visualize_phase3_enhanced(file_path):
    if not os.path.exists(file_path):
        print(f"❌ Error: File not found at {file_path}")
        return

    with open(file_path, 'r') as f:
        data = json.load(f)

    # --- 1. DATA PARSING ---
    rows = []
    pattern = re.compile(r"unsw_(.*?)(IID|NonIID_a[\d.]+)_s(\d+)")

    for exp_name, metrics in data.items():
        m = pattern.match(exp_name)
        if not m: continue
        strategy, condition, seed = m.groups()
        rows.append({
            "Strategy": strategy.upper(),
            "Condition": condition,
            "Seed": int(seed),
            "AUC": metrics.get("auc_roc", 0),
            "F1": metrics.get("f1", 0),
            "DR": metrics.get("dr", 0),
            "FAR": metrics.get("far", 0)
        })

    df = pd.DataFrame(rows)

    # Sorting logic
    def get_sort_rank(c):
        if c == 'IID': return 10.0
        match = re.findall(r"[\d.]+", str(c))
        return float(match[0]) if match else 0.0

    df['sort_rank'] = df['Condition'].apply(get_sort_rank)
    df = df.sort_values(['sort_rank', 'Strategy'], ascending=[False, True])
    sns.set_theme(style="whitegrid")

    # --- 2. BAR CHART COMPARISON (Clearer DR/FAR view) ---
    fig, axes = plt.subplots(1, 2, figsize=(18, 6))

    # Detection Rate Bar Chart
    sns.barplot(data=df, x="Condition", y="DR", hue="Strategy", ax=axes[0])
    axes[0].set_title("Detection Rate (DR) Comparison", fontweight='bold')
    axes[0].set_ylim(0, 1.0)

    # False Alarm Rate Bar Chart
    sns.barplot(data=df, x="Condition", y="FAR", hue="Strategy", ax=axes[1])
    axes[1].set_title("False Alarm Rate (FAR) Comparison", fontweight='bold')
    axes[1].set_ylim(0, max(df['FAR'].max()*1.2, 0.1)) # Scaled to data

    plt.tight_layout()
    plt.show()

    # --- 3. RADAR CHART (Overall Strategy Balance) ---
    agg = df.groupby(['Strategy']).mean(numeric_only=True).reset_index()
    # Invert FAR so that "higher is better" for all radar axes
    agg['FAR_Inverted'] = 1 - agg['FAR']
    categories = ['AUC', 'F1', 'DR', 'FAR_Inverted']

    label_loc = np.linspace(start=0, stop=2 * np.pi, num=len(categories))
    plt.figure(figsize=(8, 8))
    ax = plt.subplot(111, polar=True)

    for i, strategy in enumerate(agg['Strategy'].unique()):
        values = agg[agg['Strategy'] == strategy][categories].values.flatten().tolist()
        values += values[:1] # Close the loop
        lines = np.concatenate([label_loc, [label_loc[0]]])
        ax.plot(lines, values, label=strategy, linewidth=2)

```

```
ax.fill(lines, values, alpha=0.05)

ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
plt.xticks(label_loc, categories)
plt.title("Overall Strategy Performance Comparison", y=1.08, fontweight='bold')
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
plt.show()

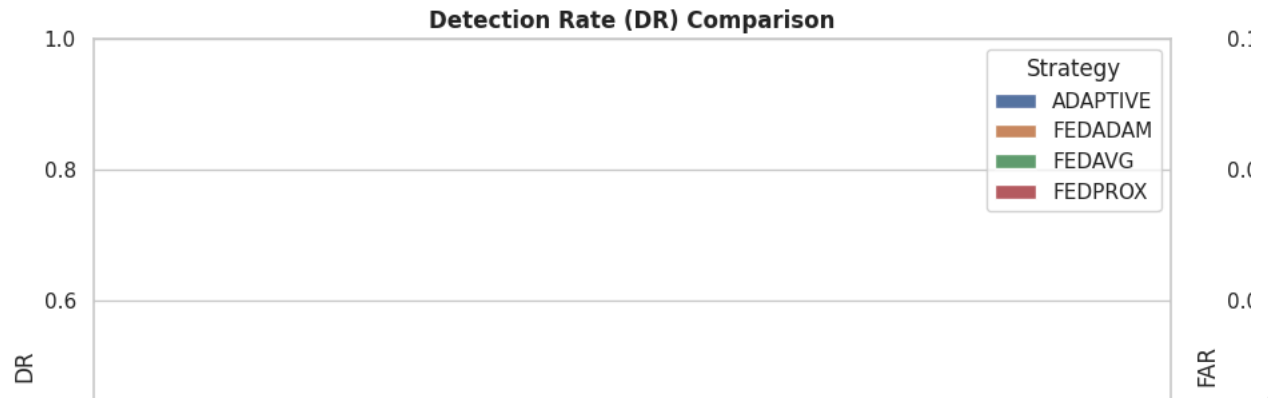
# --- 4. SCATTER PLOT WITH JITTER (Retained from previous fix) ---
plt.figure(figsize=(10, 6))
df_plot = df.copy()
jitter = (df_plot['FAR'].max() - df_plot['FAR'].min()) * 0.1 if (df_plot['FAR'].max() - df_plot['FAR'].min()) > 0 else 0
df_plot['FAR_jittered'] = df_plot['FAR'] + np.random.uniform(-jitter, jitter, size=len(df_plot))

sns.scatterplot(data=df_plot, x="FAR_jittered", y="DR", hue="Strategy", style="Condition", s=150, alpha=0.8)
plt.title("DR vs FAR Trade-off (Zoomed View)", fontweight='bold')
plt.xlim(df_plot['FAR'].min() - jitter*2, df_plot['FAR'].max() + jitter*2)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()

return agg

# Execution
json_path = "/content/drive/MyDrive/fl_anomaly_detection/results/phase3_raw_results.json"
summary_stats = visualize_phase3_enhanced(json_path)
```


Mounted at /content/drive



```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
import os
import re
from google.colab import drive

# 1. Mount Drive
try:
    drive.mount('/content/drive', force_remount=True)
except:
    pass

def load_and_clean(path, phase_label):
    if not os.path.exists(path):
        print(f"⚠ Warning: {path} not found.")
        return pd.DataFrame()

    with open(path, 'r') as f:
        data = json.load(f)

    rows = []
    # Phase 2 pattern: {strategy}_{condition}
    # Phase 3 pattern: unsw_{strategy}_{condition}_s{seed}
    for key, m in data.items():
        # Standardize strategy names and conditions
        strat = re.search(r'(fedavg|fedprox|fedadam|adaptive)', key.lower()).group(0).upper()
        cond = "IID" if "iid" in key.lower() else re.search(r'NonIID_a[\d.]+', key).group(0)

        rows.append({
            "Phase": phase_label,
            "Strategy": strat,
            "Condition": cond,
            "AUC": m.get("auc_roc", 0),
            "DR": m.get("dr", 0),
            "FAR": m.get("far", 0)
        })

    return pd.DataFrame(rows)

# 2. Paths
p2_path = "/content/drive/MyDrive/fl_anomaly_detection/results/phase2_all_results.json"
p3_path = "/content/drive/MyDrive/fl_anomaly_detection/results/phase3_raw_results.json"

# 3. Process Data
df2 = load_and_clean(p2_path, "Phase 2 (NSL-KDD)")
df3 = load_and_clean(p3_path, "Phase 3 (UNSW-NB15)")

# Average Phase 3 seeds so it matches Phase 2 structure
df3_avg = df3.groupby(["Phase", "Strategy", "Condition"]).mean().reset_index()
combined_df = pd.concat([df2, df3_avg], ignore_index=True)

# 4. Visualization
sns.set_theme(style="whitegrid")
metrics = ["AUC", "DR", "FAR"]
fig, axes = plt.subplots(3, 1, figsize=(14, 18))

for i, metric in enumerate(metrics):
    sns.barplot(
        data=combined_df,
        x="Condition",
        y=metric,
        hue="Phase",

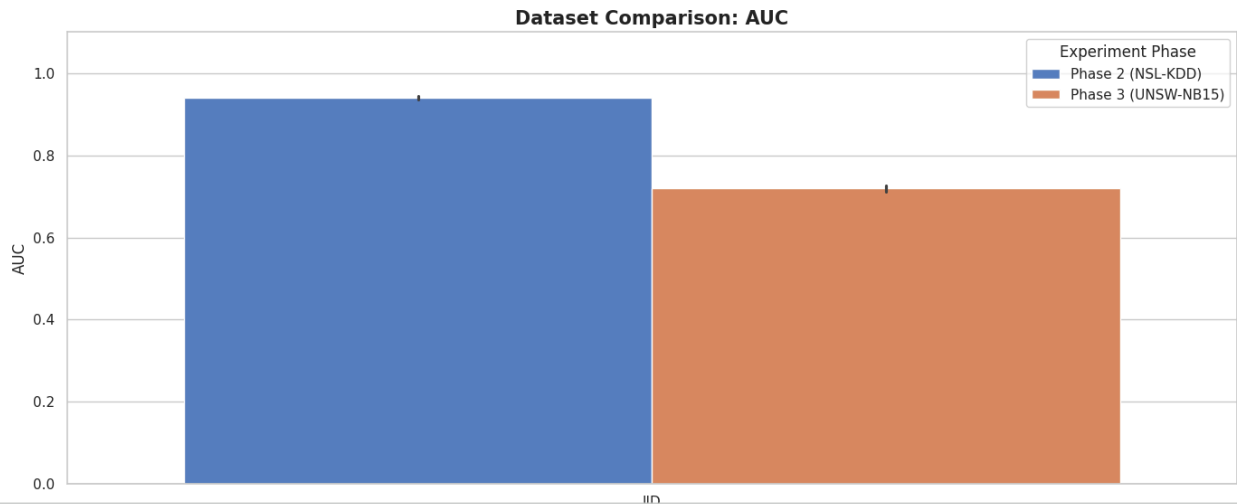
```

```
        ax=axes[i],
        palette="muted"
    )
    axes[i].set_title(f"Dataset Comparison: {metric}", fontsize=15, fontweight='bold')
    axes[i].set_ylim(0, 1.1 if metric != "FAR" else combined_df["FAR"].max() * 1.5)
    axes[i].legend(title="Experiment Phase", loc='upper right')

plt.tight_layout()
plt.show()

# 5. Summary Table
comparison_table = combined_df.groupby(["Phase", "Strategy"]).mean(numeric_only=True).drop(columns=['sort_rank'], errors='ignore')
print("\n--- Cross-Phase Dataset Performance Summary ---")
print(comparison_table)
```


Mounted at /content/drive



```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# 1. Calculate the 'Delta' (Difference) between Phase 2 and Phase 3
# This explains how well models generalize to new, harder datasets
p2_mean = df2.groupby("Strategy")["AUC"].mean()
p3_mean = df3_avg.groupby("Strategy")["AUC"].mean()
delta = (p3_mean - p2_mean).reset_index(name="AUC_Drop")
```

```
# 2. Plotting the Insights
fig, axes = plt.subplots(2, 2, figsize=(18, 14))
plt.subplots_adjust(hspace=0.3)
```

```
# --- A. STABILITY ANALYSIS (Boxplot) ---
# Explains which Strategy is most "reliable" regardless of data distribution
sns.boxplot(data=combined_df, x="Strategy", y="AUC", hue="Phase", ax=axes[0, 0], palette="Set2")
axes[0, 0].set_title("Strategy Reliability: AUC Spread", fontweight='bold')
```

```
# --- B. THE "SWEET SPOT" TRADE-OFF (Scatter) ---
# Explains model behavior: High DR is good, but only if FAR remains low.
sns.scatterplot(data=combined_df, x="FAR", y="DR", hue="Phase", style="Strategy", s=100, ax=axes[0, 1])
axes[0, 1].set_title("Efficiency Trade-off: Detection Rate vs False Alarm Rate", fontweight='bold')
axes[0, 1].set_xlabel("False Alarm Rate (Lower is Better)")
axes[0, 1].set_ylabel("Detection Rate (Higher is Better)")
```

```
# --- C. GENERALIZATION GAP (Heatmap) ---
# Explains how much "knowledge" is lost when moving from NSL-KDD to UNSW-NB15
delta_pivot = delta.set_index("Strategy")
sns.heatmap(delta_pivot, annot=True, cmap="RdYlGn", center=0, ax=axes[1, 0])
axes[1, 0].set_title("Generalization Gap: AUC Change (P2 to P3)", fontweight='bold')
```

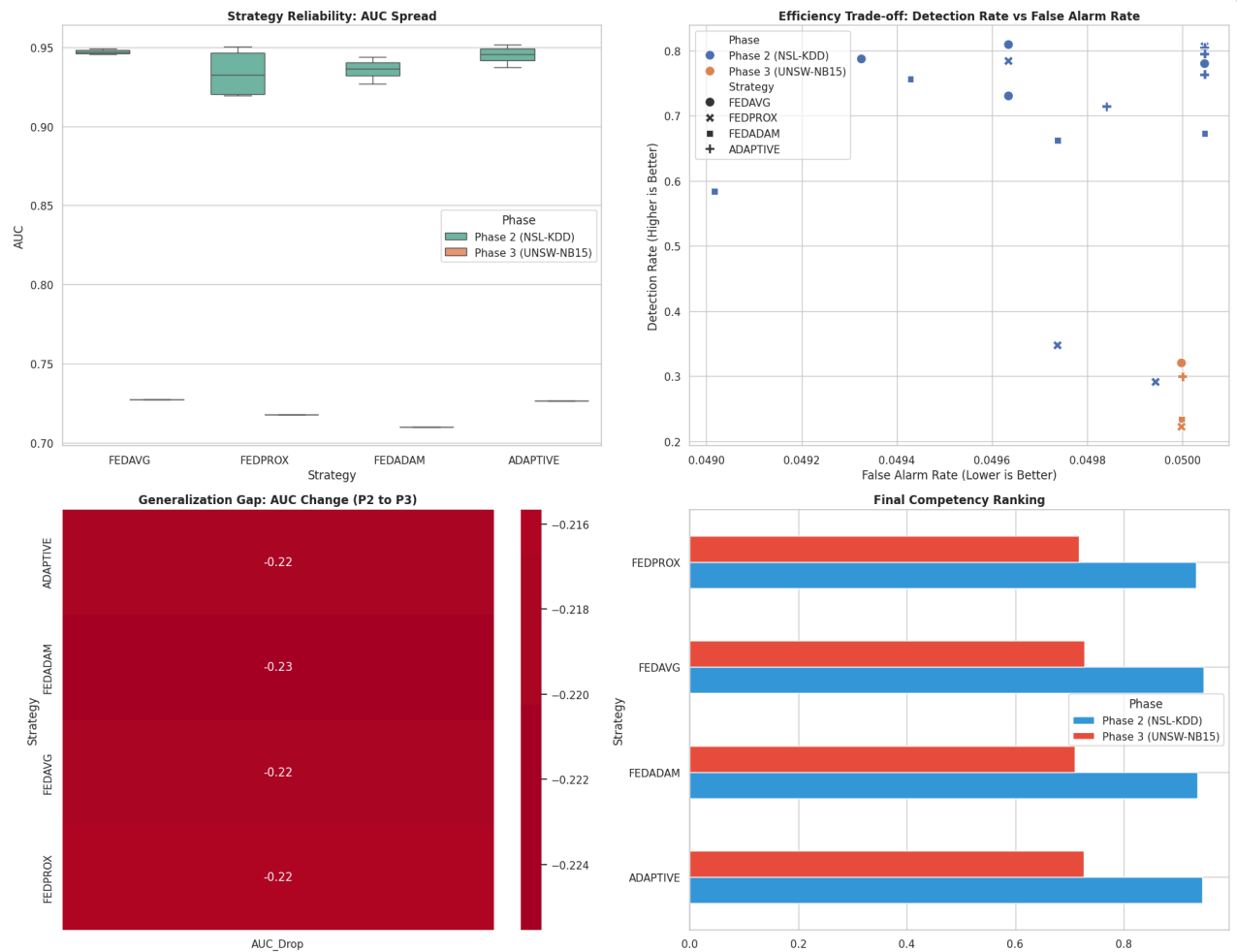
```
# --- D. PERFORMANCE RANKING ---
# A clear ranking of which models learned the most robust features
rank_df = combined_df.groupby(["Phase", "Strategy"])["AUC"].mean().unstack("Phase")
rank_df.plot(kind="barh", ax=axes[1, 1], color=['#3498db', '#e74c3c'])
axes[1, 1].set_title("Final Competency Ranking", fontweight='bold')
```

```
plt.tight_layout()
plt.show()
```

IID
Condition

```
--- Cross-Phase Dataset Performance Summary ---
```

		AUC	DR	FAR
Phase 2 (NSL-KDD)	ADAPTIVE	0.944899	0.769637	0.049995
	FEDADAM	0.935838	0.668939	0.049557
	FEDAVG	0.947054	0.776845	0.049660
	FEDPROX	0.933695	0.557274	0.049840
Phase 3 (UNSW-NB15)	ADAPTIVE	0.726675	0.299341	0.050000
	FEDADAM	0.710304	0.233297	0.049998
	FEDAVG	0.727590	0.320184	0.049998
	FEDPROX	0.718043	0.222078	0.049998



```
# =====
# CELL 9 – CLEAN Phase 2 + Phase 3 Comparison (NO SCAFFOLD)
# =====

import json, os
import pandas as pd
import numpy as np

P2_PATH = '/content/drive/MyDrive/fl_anomaly_detection/results/phase2_all_results.json'
P3_PATH = '/content/drive/MyDrive/fl_anomaly_detection/results/phase3_raw_results.json'

# =====
# Load JSON safely
# =====
def load_json(path):
    if not os.path.exists(path):
        print(f"❌ Missing: {path}")
        return None
    with open(path) as f:
        return json.load(f)

# =====
# PHASE 2
# =====
```

```

p2 = load_json(P2_PATH)

p2_rows = []
if p2:
    for exp, v in p2.items():
        p2_rows.append({
            "Dataset": "NSL-KDD",
            "Experiment": exp,
            "AUC-ROC": float(v.get("auc_roc", 0)),
            "F1": float(v.get("f1", 0)),
            "DR": float(v.get("dr", 0)),
            "FAR": float(v.get("far", 0)),
        })

p2_df = pd.DataFrame(p2_rows)

# -----
# PHASE 3 (aggregate seeds)
# -----
p3 = load_json(P3_PATH)

p3_rows = []
if p3:
    temp = {}

    for k, v in p3.items():
        if not isinstance(v, dict):
            continue

        base = "_".join(k.split("_")[:3]) # unsw_fedavg_IID etc.

        if base not in temp:
            temp[base] = {"auc": [], "f1": []}

        if "auc_roc" in v:
            temp[base]["auc"].append(float(v["auc_roc"]))
        if "f1" in v:
            temp[base]["f1"].append(float(v["f1"]))

    for base, vals in temp.items():
        if len(vals["auc"]) > 0:
            p3_rows.append({
                "Dataset": "UNSW-NB15",
                "Experiment": base,
                "AUC-ROC": np.mean(vals["auc"]),
                "F1": np.mean(vals["f1"]),
                "DR": None,
                "FAR": None
            })

p3_df = pd.DataFrame(p3_rows)

# -----
# MERGE
# -----
df = pd.concat([p2_df, p3_df], ignore_index=True)

# -----
# NORMALIZE STRATEGIES (NO SCAFFOLD)
# -----
def get_strategy(exp):
    exp = exp.lower()
    if "fedavg" in exp: return "FedAvg"
    if "fedprox" in exp: return "FedProx"
    if "fedadam" in exp: return "FedAdam"
    if "adaptive" in exp: return "Adaptive"
    return "Unknown"

df["Strategy"] = df["Experiment"].apply(get_strategy)

# -----
# CONDITION LABELS
# -----
def get_condition(exp):
    exp = exp.lower()
    if "iid" in exp: return "IID"
    if "a0.1" in exp or "alpha=0.1" in exp: return "Non-IID  $\alpha=0.1$ "
    if "a0.5" in exp or "alpha=0.5" in exp: return "Non-IID  $\alpha=0.5$ "
    if "a1.0" in exp or "alpha=1.0" in exp: return "Non-IID  $\alpha=1.0$ "

```